FP 12 (Haskell 8): Monads

Cael Marquard
mrqcae001@myuct.ac.za
(slides adapted from Dr Sabre Didi)

Aim of this lecture

By the end of this lecture you should:

- Know what `Maybe`, `List`, and `IO` have in common.
- Understand maps and flatmaps on monads
- Be able to use `do` notation
- Know why a monad is like a burrito

It is now the section on Monads

Pay attention!

`Maybe` - propagating failure

```
data Maybe a = Nothing | Just a
```

The `Maybe` type represents something that either ***can*** be there, or ***can't*** be there. Let's show how this can be used in practice. If I want to get a cat picture from the internet, I need to do the following:

1. Go to Google Search and type in "cat"
2. Get the first link
3. Load the webpage at the first link
4. Look for the first `<img>` tag
5. Download that `<img>`'s `src` and save it to disk

Maybe type - propagating failure (2)

Any one of these steps can fail:

1. Go to Google Search and type in “cat” ***(fail if eduroam is broken)***
2. Get the first link ***(fail if no results)***
3. Load the webpage at the first link ***(fail if link gives 404)***
4. Look for the first `<img>` tag ***(fail if no <img> tag)***
5. Download that `<img>`’s `src` ***(fail if src gives 404)***
6. Save it to disk ***(fail if no disk space)***

How do we express our process altogether while gracefully handling failure at any point? *Gracefully* means we don’t just crash, our final result should be `Nothing`

Maybe type - propagating failure (3)

```
searchCat :: Maybe String
searchCat = firstResult $ searchGoogle "cat"
    where
  firstResult (Just (link:links)) = Just link
  firstResult (Just []) = Nothing
  firstResult Nothing = Nothing
```

This is really long and annoying, and also I am *way* too lazy to write this out for *every function…* This is also not very elegant… **Is there a better way?**

Maybe type - propagating failure (4)

First, we can replace two patterns with one using the `listToMaybe :: [a] -> Maybe a` function (returns `Just` the first element of a list, if it's there, else `Nothing`). It is a safe version of `head`, which crashes if the list is empty.

```
import Data.Maybe -- gives us listToMaybe
searchCat :: Maybe String
searchCat = firstResult $ searchGoogle "cat"
    where
  firstResult (Just links) = listToMaybe links
  firstResult Nothing = Nothing
```

I mean… good, not *great*… We still have two patterns here in our function.

Maybe type - propagating failure (5)

What we want to do is a bit like a **map on a list**, right? We want to call a function on the element that is *inside* of the structure, and return the structure with the changed value still inside…

There is one other difference: we want to **flatten** the structure. I.e., if we had some function `map :: (a -> b) -> Maybe a -> Maybe b`, and we had our mapper function be listToMaybe, we'd end up with `Maybe (Maybe String)`... but we actually want to **map + flatten** (collapse `Just Just "cats.com"` to `Just "cats.com"`)

Flat-map, bind, >>=

We can call this operator “flat-map”, “bind”, or `>>=`.

It works exactly as we want!

```
(>>=) :: Maybe a -> (a -> Maybe b) -> Maybe b
searchCat :: Maybe String
searchCat = searchGoogle "cat" >>= listToMaybe
```

Compare its type signature with that of `fmap` (generalized map, works on lists and Maybe too!) to see why it ‘flattens’ it:

```
(>>=) :: Maybe a -> (a -> Maybe b) -> Maybe b
fmap  :: (a -> b) -> Maybe a -> Maybe b
```

Using >>= to propagate failure

We can now elegantly express our whole sequence. If *any* one of these steps fail, the entire thing stops and returns `Nothing` early. And, we don't need to bother manually handling it at each step!

```
searchGoogle "cat"             -- 1. Search “cat” on google
  >>= listToMaybe              -- 2. Get first link
  >>= loadWebpage              -- 3. Load webpage at first link
  >>= getImgs >>= listToMaybe -- 4. Get first <img>
  >>= downloadImg              -- 5. Download img at <src>
  >>= saveToDisk               -- 6. Save to disk
```

>>= for lists

We’re now going to generalise this idea.

>>= is a **flat map** for Maybe because it **collapses** `Just Just x` into `Just x`

What would this look like for a list?

E.g., what would

```
[1, 2, 3] >>= (\n -> [1..n])
```

return?

>>= for lists (2)

```
[1, 2, 3] >>= (\n -> [1..n])
```

… is telling us to map the `(\n -> [1..n])` function over the list…

```
[[1], [1, 2], [1, 2, 3]]
```

… and then flatten it!

```
[ 1,  1,  2,   1, 2, 3  ]
```

So, this is our final list.

Genericizing >>=: Monads

So, we can use `>>=` for lists, and also for `Maybe`.

As we learned in our lecture on polymorphism, we know that >>= must be a **generic function,** and that both `List` and `Maybe` must both implement a **common typeclass.**

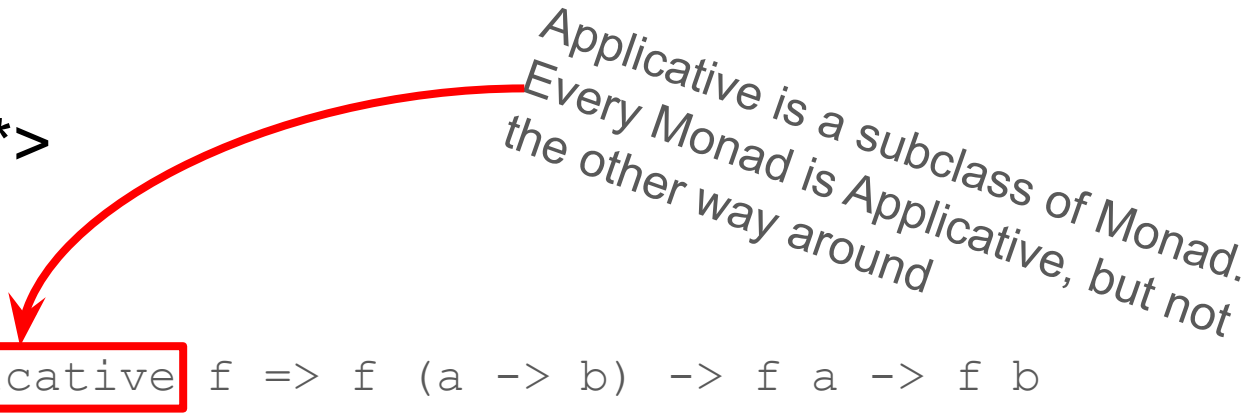
We call this typeclass a *Monad*. It turns out that we can do a lot more with them than just flat map!

What else can we do with Monads?

Alongside >>=, a monad comes with a couple other useful operations:

- **`pure`**, which makes an instance of a monad out of a value
  - e.g. `pure x = [x]` for lists, `pure x = Just x` for Maybe.
- **`fmap fn mon`**, which applies the given function to the 'inside' of the monad
  - This is ***exactly*** equivalent to `map` for lists (literally fmap is defined for lists using map)
- **<$>** exactly the same as **fmap** but as an infix
  - E.g. `(*2) <$> myList = map (*2) myList`
- **<*>**, which sequences computations and can be used to curry functions in monads
  - E.g. `(*) <$> Just 1 <*> Just 3 = Just (1 * 3)`

A closer look at <*>



<*> has the type:

```
<*> :: Applicative f => f (a -> b) -> f a -> f b
```

Basically, it takes the function *inside* our wrapper, and applies it to the *value* in the wrapper, returning new wrapped value. **It is like fmap / <$> for when your function is also wrapped with `Maybe` / `List`.**

<*> can be used to express function currying with monads:

```
(*)                          :: Int -> Int -> Int

(*) <$> Just 1               :: Maybe (Int -> Int)

(*) <$> Just 1 <*> Just 3 :: Maybe Int
```

Monads as *effect systems*

Let’s look back at our cat picture example from before…

```
searchGoogle "cat"            -- 1. Search “cat” on google
  >>= listToMaybe             -- 2. Get first link
  >>= loadWebpage             -- 3. Load webpage at first link
  >>= getImgs >>= listToMaybe -- 4. Get first <img>
  >>= downloadImg             -- 5. Download img at <src>
  >>= saveToDisk              -- 6. Save to disk
```

Could this be writable with just fmap?

Monads as *effect systems* (2)

```
searchGoogle "cat"            -- 1. Search “cat” on google
  >>= listToMaybe             -- 2. Get first link
  >>= loadWebpage             -- 3. Load webpage at first link
  >>= getImgs >>= listToMaybe -- 4. Get first <img>
  >>= downloadImg             -- 5. Download img at <src>
  >>= saveToDisk              -- 6. Save to disk
```

NO! It’s clear even from the signature that fmap **only lets us change what is INSIDE the structure:**

```
fmap :: Functor f => (a -> b) -> f a -> f b
```

(or if we write it for Maybe only)

```
fmap :: (a -> b) -> Maybe a -> Maybe b
```

At no point do we get to say “wait, something is broken, abandon ship!”

Monads as *effect systems* (3)

Monads give us >>=, which is something that just `Applicative`s lack.

It's clear again from the signature that >>= **DOES lets us change the structure itself:**

```
(>>=) :: Monad m => (a -> f b) -> f a -> f b
```

This is because we **return a monad from our mapping function**, which then gets **combined** (flattened) **with the original function.**

Monads as *effect systems* (4)

I’ve shown you how this works with chaining >>= on Maybe, but let’s look at lists too.

Can we modify a list with `fmap` and add elements to it?

Monads as *effect systems* (5)

NO! This is clear from the type signature of fmap. Here I've simplified it to work on only lists:

```
fmap :: (a -> b) -> [a] -> [b]
```

Or, if it only worked on Int lists:

```
fmap :: (Int -> Int) -> [Int] -> [Int]
```

We only ever can return an Int. We can't return more than one or less than one, so the total number of Ints MUST stay the same!

So, how could we implement **filter** on a list, thinking with monads???

Monads as *effect systems* (6)

We can use >>= to implement filter like this, thinking of it like a flatmap:

```
filter f xs = xs >>= (\x -> if f x then [x] else [])
```

We could also use >>== to repeat every value twice in a list:

```
repeat2 xs = xs >>= (\x -> [x, x])
```

In both cases, we are doing more than just modifying the values *inside* the structure. We affect the structure itself! This is an effect.

Monads as *effect systems* (7)

We can even use monads to track effects more directly. For example, we could keep a count of how many times we've called `inc` from inside out monad:

```
data CounterMonad a = Count Int a deriving Show
instance Monad CounterMonad where
    Count times val >>= f = Count (times + times') val'
      where Count times' val' = f val
instance Applicative CounterMonad where
    pure = Count 0
    Count times f <*> Count times' val = Count (times + times') (f val)
instance Functor CounterMonad where
    fmap f (Count times val) = Count times (f val)

inc = Count 1 ()
x  = Count 0 "Hi"                          -- returns Count 0 "Hi"
x' = (++ " there") <$> x <* inc <* inc <* inc -- returns Count 3 "Hi there"
```

`CounterMonad` watching me `inc`



Monads as *effect systems* (8)

Because we defined the monad, and our monadic interfaces (<$>/fmap, <*>, and >>=), we can **play gatekeeper** and see any time that `inc` gets called and log it

```
x'   = (++ " there") <$> x -- returns Count 0 "Hi there"
x''  = x *> inc            -- returns Count 1 "Hi there"
```

In a similar way, this is how IO works in Haskell. **The Haskell compiler uses the IO monad to play gatekeeper to our IO actions! You could imagine the "inc" function being replaced with something that tells ghc to actually go and do some IO…**

Monads as *effect systems* (9)

You can imagine a `DummyIO` monad which, instead of actually doing IO, it just stores a **list of instructions for IO things that we will one day do…**

```haskell
data Action = Print String
    | Write { fname :: String, text :: String }
    | Delete { fname :: String } deriving Show

data DummyIO a = Dummy [Action] a deriving Show
instance Monad DummyIO where
    Dummy actions val >>= f = dummy'
      where Dummy actions' val' = f val
            dummy' = Dummy (actions ++ actions') val'

-- applicative, functor impls left for the reader

x = Dummy [] "Hi"
-- returns Dummy [] "Hi"
```

```haskell
done = do
   x' <- x
   let out = x' ++ " there"

   print' out
   delete "somefile.txt"
   write "out.txt" out

   pure out
-- Dummy [
          Print "Hi there",
        , Delete {fname = "somefile.txt"},
        , Write {fname = "out.txt", text = "Hi there"}
    ] "Hi there"
```

Haskell compiler watching me do IO through the `IO` monad



Monads as *effect systems* (10)

We won't cover what an **effect system** is formally, but all you need to know is this:

- Monads wrap values, but **they carry some kind of higher information and structure** (you could also think of this as *similar to* metadata). E.g., a Maybe carries "presence or not presence", List carries "there are multiple values".
- We can change the values only with `fmap`.
- We can also change the **monad structure** with >>= / bind / flatmap

Know what else could be represented as "higher information" / "metadata" about a function? **WHETHER IT DOES I/O!**

The IO monad

Let's say we have a function that **reads a line from the terminal** (so, the same as `input()` in Python) and returns it

```
getLine :: String
```

If we define this function just as-is, Haskell doesn't know if we're doing IO here (we are!), so it will assume that `getLine` is therefore, a pure function, and we can **substitute the invocation for its value**…

But this would not work! If we call it twice, the user could input different things, so we **CANNOT substitute the invocation for its value. It MUST be run again!** How do we let Haskell know this?

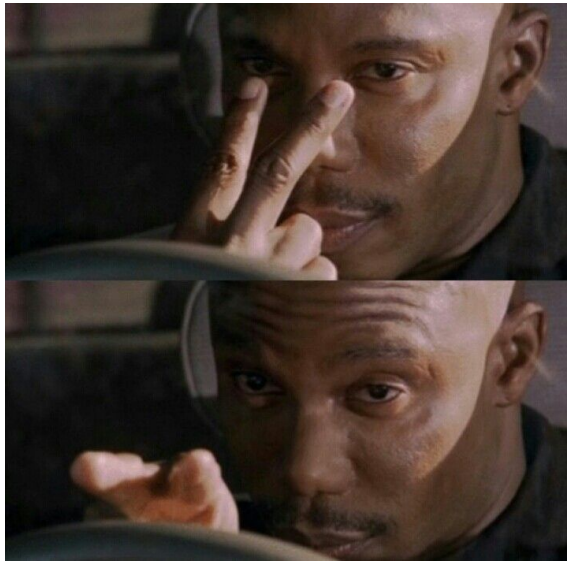
The IO monad (2)

We can **wrap** our **String** return type in a special monad called **IO**

```
getLine :: IO String
```

IO is a monad whose metadata / **higher structure tracks the IO actions we perform** (like Maybe tracks if the value is there or not, and list tracks multiple values).

It also signals to the compiler that our function is not pure and we can’t just replace it by its value.



Now GHC knows…

>>= in the `IO` monad

So, if `IO` is a monad, what does >>= do?

>>= does exactly the same thing as in other cases. It maps the value and *merges the structure/metadata*. What that means is that you map the value inside the IO, and **merge whatever IO that function does into the same IO type.**

For example, if we do

```
putStrLn "Enter name" >>= \_ -> getLine >>= putStrLn . ("Hello, " ++)
```

We throw away the () from the inside of putStrLn's IO (), replacing it with getLine's IO String. But, we **merge in the *IO effect of printing!* Even though we threw away the value, THE EFFECT ON THE STRUCTURE REMAINED!**

>>: throwing away results

We've seen that >>= flatmaps the value. You produce the new value with the old one. >> is similar, but it **merges structure, *replacing* the value** without using the old as input.

```
x :: IO () = putStrLn "Hi"

x >>= (\_ -> putStrLn "There")
-- is the same as
x >> putStrLn "There"
```

>> is actually the same as *> from earlier, but *> is more general (it's from Applicative. All Monads are Applicative, but not all Applicatives are Monads)

do notation

Haskell has a nice notation we can use to work with Monads called do notation. It is **purely syntactic sugar** (i.e. any do block can be implemented in terms of >>=).

```
greet :: IO String
greet = do
    putStrLn "What is your name?"
    name <- getLine
    let greeting = "Hello, " ++ name ++ "!"
    putStrLn greeting
    pure name

greet >>= putStrLn . ("Return value was " ++)
```

do notation (2)

```
greet :: IO String
greet = do
    putStrLn "What is your name?"
    name <- getLine
    let greeting = "Hello, " ++ name ++ "!"
    putStrLn greeting
    pure name

greet >>= putStrLn . ("Return value was " ++)
```

1. Every statement must be:
   a. A let
   b. A <- expr
   c. An instance of the monad
2. <- is used to ‘unwrap’ the monad (e.g. get the String from IO String)
3. let is used to make temporary variables
4. pure is used to ‘wrap’ a value into a monad

Notes on do notation

do notation is the *closest* Haskell gets to being imperative, but it's still different. Some important notes:

1. **You can't modify variables with `lets`.** If I define let x = x + 2, **this will loop forever** (you're recursively defining x in terms of x…)
2. **You *can* modify variables with <-.** This works as you may expect.
3. **Every statement in a do block must be a monadic value, a <- expression, or a let expression.**

Things I’m not covering but are useful (not examinable)

- MonadPlus: monads which can have default, empty values (Maybe, List, IO)
  - Can be filtered with a general function
- MonadFail: monads that have failure states.
  - These can have *fallible patterns* in their do block <- expressions, which is quite convenient! When a pattern match would fail, the do block ends early, combining the failure into the monad’s structure
- Other Applicative / Monad functions
  - <$, $>, <*, *>, &, >>> (>> is in scope, >>> is something else!)
- Differences between Functors, Applicatives, and Monads

Exercises - longer because Monads can be HARD!

(please remember that if you use AI to do the following, it no longer has any value. You are only cheating yourself.)

- Implement `Monad` for a custom type representing computations that might error. This is similar to Maybe, but it should *store* its error as a value. Define it as `data Result e v = Ok v | Err e`. You'll need to implement Functor and Applicative too, of course. **HINT:** `instance Functor (Result e) where...`
- Implement a `throw :: e -> Result v e` function which represents an error happening in some computation.
- Implement an `orErr :: Maybe v -> e -> Result v e` function that returns the given error if a given `Maybe` is `Nothing`.
- Write a short do function that takes the user's name (String) and age (String). `throw` an error if their name is less than 5 characters. Use the `readMaybe` function to convert their age to an Int, and use orErr to return an error if it fails.

Assigned reading

Revision for today:

- https://www.haskell.org/tutorial/monads.html
- https://wiki.haskell.org/Merely_monadic

Preparation for tomorrow:

- https://functional-architecture.org/make_illegal_states_unrepresentable/
- “Parse, don’t validate” by Alexis King